

Format for Assignment SubmissionFor Term End Exam June / December-December(Year) 2025

(Please read the instructions given below carefully before submitting assignments)

1. Name of the Student. : DHEERAJ KUMAR YADAV
2. Enrollment Number : 2450499960
3. Programme Code : BCA-NEW
4. Course Code : MCS-208
(Use this format course-wise separately)
5. Study Centre Code : 07162P
6. Name of the Study Centre : MODHYAL EDUCATION AND RESEARCH
With complete address : INSTITUTE OF TECHNOLOGY, NEW DELHI
7. Mobile Number : 9570448220
8. E-mail ID : kumardheeraj8486@gmail.com
9. Details if this same assignment has : NO
been submitted anywhere else also
10. Above information is cross checked and it is correct: Yes/No

Date of Submission: 8th October, 2025Dheeraj kr. yadav
(Signature of the student)**A. Important Instructions:-**

1. Please do not send any assignment at any email of the Regional Centre, it will not be considered.
2. Please avoid duplicacy. Do not re-submit the same assignment anywhere else or by any other means.
3. About the mode of submission of assignments, pl wait for instructions from IGNOU Hqtrs. As soon as we shall come to know, we will share it with all.
4. Please do not use plastic covers. Use plain A4 size pages for assignments for uniformity and better management with this cover page format on each assignment.
5. Please write your name and enrollment no. at the bottom of each page of your assignment.
6. Please retain a photocopy set of assignment submitted with you for record (may be asked to submit at later stage) and also keep the assignment submission receipt in safe custody.
7. If assignment awards are not updated in your Grade Card within next 09 months, please write to us at rcdelhi1@ignou.ac.in giving your complete details and attaching the proof of assignment submission.
8. Assignment Question Paper can be downloaded from: <https://webservices.ignou.ac.in/assignments/>

B. Compulsory sequence of the Assignment Set:

1. Duly Filled in Assignment Submission Cover Page (This Format Page).
2. Copy of IGNOU Identity Card.
3. Printout of valid/applicable assignment question paper.
4. Handwritten Assignment, written on both the sides of page (preferably plain A4 size).



इंदिरा गांधी राष्ट्रीय मुक्त विश्वविद्यालय
मैदान गढ़ी, नई दिल्ली - 110068
Indira Gandhi National Open University
Maldan Garhi, New Delhi - 110068



IGNOU - Student Identity Card

Enrolment Number : 2450499960

RC Code : 07: DELHI 1 (MOHAN ESTATE (SOUTH DELHI))

Name of the Programme : BCA_NEW : Bachelor of Computer Applications

Name : DHEERAJ KUMAR YADAV

Guardian's Name : LAKHAN YADAV

Address : VILLAGE NAITAND, POST PATHALDIHA
KODERMA JHARKHAN JHARKHAND

Pin Code : 825410



2450499960

Dheeraj k yadav

Instructions:

1. This card should be produced on demand at the Study Center, Examination Center or any other Establishment of IGNOU to use its facilities.
2. The facilities would be available only relating to the Programme/course for which the student is registered.
3. This ID Card is generated online. Students are advised to take a color print of this ID Card and get it laminated.
4. The student details can be cross checked with the QR Code at www.ignou.ac.in. PD 2025-04-30

Registrar
Student Registration Division

Course Code	:	MCS-208
Course Title	:	Data Structures and Algorithms
Assignment Number	:	BCA_NEW(I)-208/Assignment/2025-26
Maximum Marks	:	100
Weightage	:	25%
Last Dates for Submission	:	31st October,2025(For July 2025 Session) 30th April, 2026 (For January 2026 Session)

There are four questions in this assignment, which carry 80 marks. Each question carries 20 marks. Rest 20 marks are for viva voce. All algorithms should be written nearer to C programming language. You may use illustrations and diagrams to enhance the explanations, if necessary. Please go through the guidelines regarding assignments given in the Programme Guide for the format of presentation.

Question 1: For each of the Singly Linked List, Circularly Singly Linked List, Doubly Linked List, Circularly Doubly Linked List, write one application that is exclusively suitable for that list. For example, X may be an application for whose implementation, only Circularly Singly Linked List is suitable and others are not suitable. Justify your answer.

Question 2: We can test whether a node 'm' is a proper ancestor of a node 'n' by testing whether 'm' precedes 'n' in X-order but follows 'n' in Y-order, where X and Y are chosen from {pre, post, in}. Determine all those pairs X and Y for which this statement holds.

Question 3: Explain Left Leaning Red Black Trees. What are their advantages and disadvantages?

Question 4: Write a short note on the recent developments in the area of finding minimum cost spanning trees.

Question 1 :-

For each of the Singly Linked List, Circularly Singly Linked List, Doubly Linked List, Circularly Doubly Linked List, write one application that is exclusively suitable for that list. For example, X may be an application for whose implementation, only Circularly Singly Linked List is suitable and other are not suitable. Justify your answer.

Answer :-

Applications suitable for different types of linked lists :-

In data structure, Linked Lists are widely used for dynamic memory management and efficient insertions or deletions. However each type of linked list serves different purposes and is more suitable for specific application. Below are the ideal applications for Singly linked list, Circular Singly list, Doubly linked list and circular doubly linked list, along with justification for only why each structure is most appropriate choice.

(i) Singly linked list - Implementation of stack -

A stack is a data structure where elements are inserted (Pushed) and deleted (popped) only from one end called (top). It is perfect for implementing a stack because:

- It allows efficient insertions and deletions at beginning which corresponds to the top of stack.
- It uses less memory since it stores only one pointer.
- The structure grows dynamically as elements are added, without the need for resizing (unlike array).
- The operation like Push() and pop() have constant time complexity $O(1)$, which ensures optimal performance.

(ii) Circular Singly linked list -

Round Robin scheduling in OS.

It is best suited for implementing the Round Robin scheduling used by operation systems for time-sharing system. The reasons are:

- The circular structure allows continuous travel of the list without needing to reset the pointer or restart loop manually.

- Each process in the queue get an equal share of CPU in rotation, and once the last process is served.
- Singly linked list structure uses minimal memory, which is essential for systems with limited resources.
- It avoids the overhead of maintaining back pointers or managing NULL pointers at end of the list.

(iii) Doubly linked list —

Navigating in web Browsers.

When a user navigates through web browsers, they might want to move back and forth through their history. A doubly linked list is ideal for this because:

- It provides two-way traversal, allowing easy access to both previous and next pages.
- Operation such as moving back or forward can be performed in constant time.
- It supports dynamic insertion of new pages between visited one when the user branches to a different path in history.

(iv) Circular Doubly Linked List —

Music playlist in media player.

A circular doubly linked list is ideal for managing playlists in multi-media application where:

- Users can move to the previous or next song/video using both prev. and next links.
- The circular structure allows the playlists to loop back to the beginning when the end is reached, providing an infinite playing experience.
- Deletion or addition of a track can be done efficiently in $O(1)$ time, whether at the start, middle, or end.
- The doubly-linked list nature allows smooth bidirectional movement, improving user experience.

Conclusion :-

Each type of linked-list has its exclusive advantages based on structure and pointer usage. Their suitability for particular application as follows :-

- Singly linked list :- Stack implementation due to efficient LIFO operation.
- Circular Singly linked list :- Round Robin scheduling due to bidirectional access.

- Doubly Linked List :- Browser navigation due to bidirectional access.
- Circular Doubly Linked List :- Multimedia players due to both cyclic and two-way traversal.

Question 2:-

We can test whether a node 'm' is a proper ancestor of a node 'n' by testing whether 'm' precedes 'n' in X-order but follows 'n' in Y-order, where X and Y are chosen from (pre, post, in). Determine all those pairs X and Y for which this statement holds.

Answers :-

Determining pairs (X, Y) for testing proper ancestor using Tree Traversals.

In tree data structures, especially binary trees, we often need to test relationships between nodes - for example, whether one node is a proper ancestor of another. One powerful way to determine such relationships is by examining the traversal order of the trees.

Let's consider three common depth-first traversal orders: Preorder, Inorder and

Postorder, usually abbreviated as pre, in and post respectively. The question asks for which pair (x, y) chosen from $\{pre, in, post\}$ does the following condition determine a proper ancestor relationship?

(i) Understanding the Meaning of "proper Ancestor".

A proper Ancestor of a node ' n ' is any node that is an ancestor of ' n ' but not equal to ' n ' itself. This means it lies on the path from the root to the parent of ' n '.

To determine this through traversal, we must understand how each traversal works.

- Pre-order: Visit node $\rightarrow$ left subtree $\rightarrow$ Right subtree.
- In-order: left subtree $\rightarrow$ Visit node $\rightarrow$ Right subtree.
- Post order: left subtree $\rightarrow$ Right subtree $\rightarrow$ visit node.

(ii) Analyzing All pairs (x, y)

a.) (Pre, in)

- If n appears before n is preorder (x) and after n is inorder

- This does not always indicate m is proper ancestor.
- for e.g. - in some tree, siblings may appear like that.
- Not reliable.

b.) (Pre, Post)

- If m is before n in preorder, and after n is postorder.
- This is valid way to determine a proper ancestor.
- Because in preorder, ancestor are visited before descendants.
- In postorder, descendants are visited before ancestors.
- So, if $m < n$ in pre and $m > n$ in post, m is definitely a proper ancestor.

c.) (Pre, Pre)

- Same order $\rightarrow$ both compared in same direction
- If m comes before n in preorder, it could be an ancestor.
- Cannot conclude definitively
- Not reliable.

d.) (Pre, post)

- Already discussed $\rightarrow$ Valid pair.

e.) (in, pre)

- If m is before n in inorder and after in preorder, then.
- This does not always indicate a proper ancestor.
- Could just be sibling or unrelated node on the same link.
- Not reliable.

f.) (in, in)

- Both in same order.
- Cannot test ancestor properly.
- Not reliable.

g.) (in, post)

- If m appears before n in inorder, after in postorder.
- Again, this could happen between sibling too.
- Not reliable indicator of proper ancestor.

h.) (post, pre)

- If m is before n in postorder and after preorder.
- Doesn't indicate ancestor definitively.
- Not reliable.

(iii) Conclusion : Valid (x, y) Pairs.

After testing all combination, we find only one valid pair that satisfies the condition :

$(x, y) = (\text{Pre}, \text{Post})$

Meaning -

If node m appears before node n in preorder and node m appears after node n in postorder, then m is a proper ancestor of n .

This is because preorder traversal naturally visits a node before its descendants and postorder visits it again after all its descendants. Hence, this combination properly captures the ancestor-descendant relationship.

The only pair (x, y) for which the statement holds true is $(\text{pre}, \text{post})$.

Question 3 :

Explain Left leaning Red Black Trees. what are their advantages and disadvantages?

Answer :-

Left leaning Red-Black Trees:- It is a simplified variation of the standard Red-Black Tree, which is a type of self-balancing binary search tree. These trees are particularly used to maintain ordered data efficiently and support operations like insertion, deletion and search in $O(\log n)$ time.

LLRB trees were introduced to make the implementation of Red-Black trees easier and more instructive while maintaining similar performance. The key difference in LLRB trees is the strict rule that red links always lean to the left, thereby simplifying rebalancing during insertion and deletion.

→ Basic properties of LLRB Trees.

- i) Every node is either red or black.
- ii) The root is always black.
- iii) Red links always lean to the left.

iv) No nodes has two red links connected to it.

v) Every path from the root to a null node has the same number of black links.

→ Interpretation as a 2-3 Tree.

The left leaning Red-Black Tree can be viewed as a representative of a 2-3 tree using binary tree structure:

- A black node represents as 2-node
- A red link between two nodes represents a 3-node, where the red links leans to the left.

→ Advantages of LLRB Trees:—

i) Simple implementation: LLRB trees avoid the complexity of handling both left and right-leaning red links by only allowing red links to lean left.

ii) Balance guarantee: LLRB trees maintain perfect Black balance, ensuring logarithmic time complexity for search, insert and delete operations.

iii) Efficient Search Performance: Since the tree remain approximately balanced, search operations are fast and predictable.

iv) Imitates 2-3 trees: LLRB trees internally model the well-balanced 2-3 tree structure, making them reliable for maintaining sorted data.

v) Use in Red-world system: LLRB trees are used in application such as java's TreeMap and c++ map containers because of their efficiency and balance.

→ Disadvantages of LLRB Trees:-

i) Complex Deletion process:- Deletion logic in LLRB trees is more intricate compared to AVL trees or standard binary search trees, requiring careful handling of red links and rotations.

ii) Redundant Rotation: During insertion or deletion, sometime more rotation and color flips may be needed compared to other trees like AVL, worst case scenario.

iii) Overhead in Rebalancing: The need for continuous checking and adjusting red/black link during insertion and deletion introduces a small overhead.

iv) Memory Usage: Each node stores an additional bit for color, which might slightly increase memory usage compared to simple balanced binary search tree.

→ Conclusion :

Left-leaning Red-Black trees provide an elegant and efficient way of maintaining balanced search trees. They simplify the logic of traditional red-black by enforcing the left leaning rule and still maintain logarithmic time complexity for all basic operations. While the deletion process can be complex, the overall performance, especially in insert-heavy application, powerful choice in practical programming scenarios.

Question 4 :

write a short note on the recent developments in the area of finding minimum cost spanning trees.

Answer :—

In the field of computer Science and data structure, finding a minimum cost spanning trees (MST) of a connected, weighted undirected graph is fundamental problem. The goal is to connect all the vertices of graph using a subset of edges, such that the total edge weight is minimum and no cycles are formed. classical algorithm like Kruskal's Algorithm and Prim's Algorithm have been used

effectively for decades. However, with the increasing complexity and size of data network, as well as the demand for faster and more efficient Algorithm many recent developments have emerged in their area.

(i) faster Algorithm Using Advanced data structure:

Modern algorithm have improved upon the time complexity of traditional M&T approaches by using sophisticated data structure.

- **Fibonacci Heaps:** when used with prim's algorithm, fibonacci heaps reduce the time complexity to $O((E+V \log V))$. This makes it highly suitable for sparse graphs.

- **Soft Heaps:** These are approximate priority queues that allows error in keys but helps speed up M&T computation to linear time $O(m \alpha(m, n))$, where α is the inverse Ackermann function, which grows extremely slowly.

(ii) Parallel and Distributed M&T Algorithm:

with the rise of multi-core processors and distributed computing, researches have developed parallel algorithm for M&T that can run on multiple processors simultaneously :-

- **Boruvka's Algorithm**: It is inherently parallel and forms the basics of many parallel MST algorithm. It repeatedly adds the shortest edges from each component until a spanning tree is formed.
- **MapReduce-based Algorithm**: These are designed for large-scale distribution computing environments like Hadoop. It allow processing of massive graphs that do not fit into memory.

(iii) Dynamic MST Algorithms:

In ~~red~~ real-life scenarios, such as maintenance, transportation system and communication networks, the graph is not static - it changes over time.

- **Dynamic Trees (link-cut trees)**: These are advanced data structure used to maintain MST efficiently under dynamic conditions.
- **Federickson's Algorithm**: It offers a method to update MST in sublinear time when small changes occur in the graph.
- **Lippstein's sparsification**: It reduce the number of edges in the graph while preserving essential connectivity leading to faster update in MST.

(iv) Approximation and Randomized Algorithms: In case where exact solution are not feasible due to time constraints, approximation algorithms are used.

- Randomized Algorithm: It makes use of randomness to provide good enough solution in a very short time. They are particularly useful in streaming and sensor networks.
- Greedy Approximation Techniques: variants of Prim's and Kruskal's method are used within heuristics to quickly find near-optimal solution.

(v) Quantum and AI-based Approaches:

- Quantum Algorithm: with the rise of quantum computing, researchers are exploring algorithms based on quantum annealing and quantum walk that might solve M&T problems.
- AI and Machine Learning Models: AI is being trained to recognize patterns in graph structure and predict edges that are likely to be part of the M&T.

Conclusion :-

The field of minimum cost spanning tree has seen significant advancements beyond the traditional Kruskal and Prim algorithms. Modern development now focus on speed, scalability and adaptability to dynamic and large datasets. As technology continues to evolve, we can expect more efficient methods, especially using parallel computing, quantum technologies and AI-driven heuristics to handle increasingly complex real-world application.